Lab No. 04

CPU Scheduling — FCFS, SJF/SRTF, Round Robin (with advanced twists)

Submission Instructions

- Submit:
    - Code files: fcfs_priority.c, sjf_srtf.c, rr_cs.c
    - One PDF report with: inputs, **Gantt charts**, tables of **WT/TAT/RT**, averages, and a short comparison paragraph for each task.
    - Screenshots (or copy-paste) of terminal runs.

Objective

Equip students to implement, analyze, and compare CPU scheduling policies with realistic constraints (arrival times, preemption, context switching, and fairness).

Outcomes

By the end, students can:

- Compute **Waiting, Turnaround, and Response** times.
- Implement and compare **FCFS**, **SJF/SRTF**, and **Round Robin**.
- Model **context-switch cost**, show **Gantt charts**, and discuss **fairness/throughput**.
- Validate results with unit-like checks and handle edge cases (ties, idle CPU, late arrivals).

Pre-Lab (Reading)

- Silberschatz et al., *Operating System Concepts* — CPU Scheduling chapter.
- Understand: **arrival time**, **burst time**, **preemption**, **ready queue**, **context switch**, **aging**, **starvation**.

🔑 Useful Concepts (Quick Ref)

1. **Waiting Time (WT)**
   👉 How long a process waits in the **ready queue** before it gets CPU.
   Formula:

   WT=(Start of first CPU execution−Arrival time)+(any extra waiting between CPU bursts)

Easy meaning: Total time a process spends just waiting.

---

2. **Turnaround Time (TAT)**
   👉 Total time taken from **arrival** until the process **finishes**.
   Formula:

   TAT=Completion time−Arrival time

Easy meaning: How long it takes to get the full job done (waiting + execution).

---

3. **Response Time (RT)**
   👉 Time between **arrival** and the **first time it starts using CPU**.
   Formula:
4. RT=Firststart time−Arrival time

Easy meaning: How long a process waits before it gets the CPU for the **first time** (important for interactivity).

---

4. **Throughput**
   👉 Number of processes finished per unit of time.
   Formula:

   Throughput= Total completed processes / Total time

Easy meaning: Speed of the system — how many jobs it completes in a given time.

---

5. **CPU Utilization**
   👉 How busy the CPU is (not sitting idle).
   Formula:

CPU Utilization= (CPU busy time/Total time)×100%

Easy meaning: Percentage of time the CPU is actually working on processes.

-

---

2) Warm-Up Activities (Practice before grading)

Warm-Up A — Manual Gantt & Metrics

Given 4 processes:
P1(AT=0,BT=8),

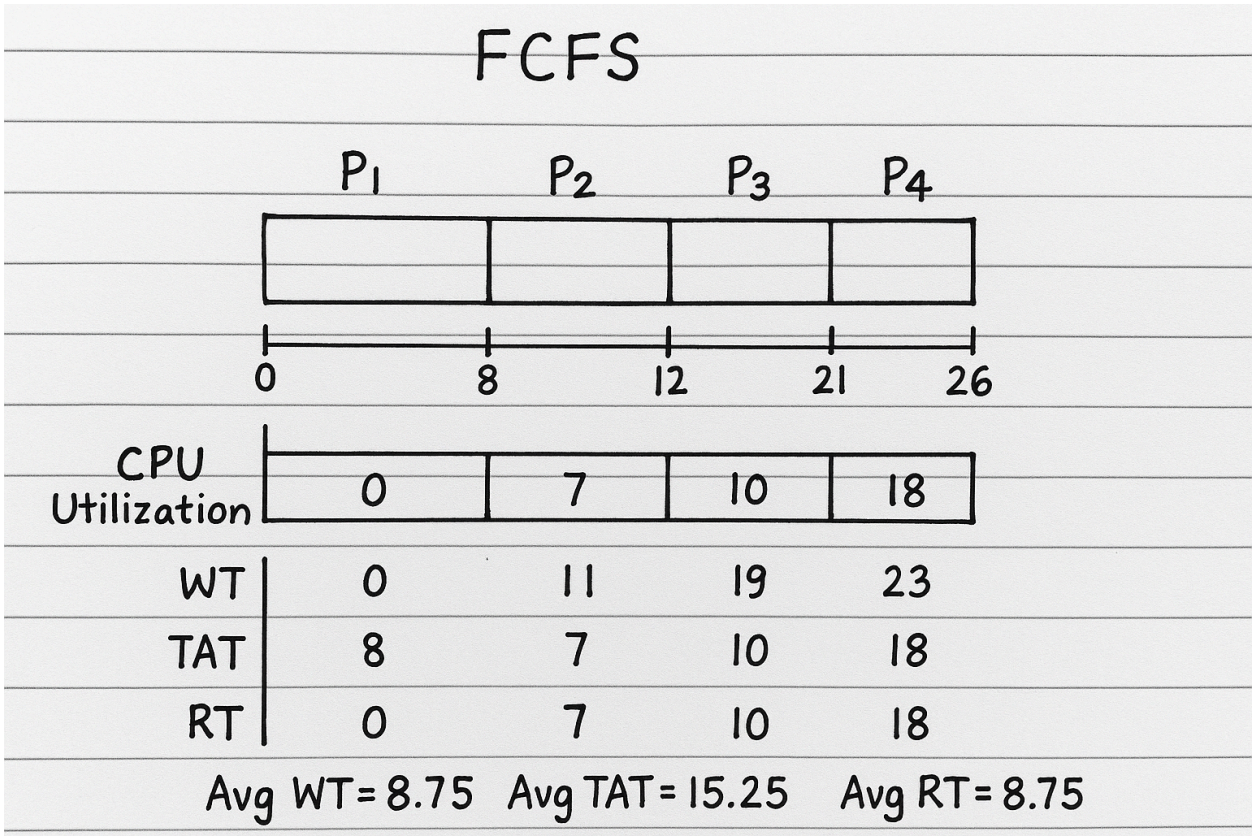
P2(AT=1,BT=4),

P3(AT=2,BT=9),

P4(AT=3,BT=5)

1. **Draw FCFS Gantt chart.**
2. **Compute Avg WT, Avg TAT, Avg RT.**

   Solution:



Warm-Up B — SJF vs FCFS intuition

Using same set, predict which metric improves most under SJF(non-preemptive): WT, TAT, or RT?

Solution:

SJF (non-preemptive) using same set

Processes:
P1(AT=0,BT=8), P2(AT=1,BT=4), P3(AT=2,BT=9), P4(AT=3,BT=5)

SJF decision: CPU starts at t=0 → only P1 is available so P1 runs (non-preemptive).
When CPU becomes free at t=8, ready set = {P2(4), P3(9), P4(5)} → pick shortest = **P2**, then **P4**, then **P3**.

Time -> 0 8 12 17 26

|----|----|----|----|

Process: P1 P2 P4 P3

(0-8)(8-12)(12-17)(17-26)

Per-process times (SJF)

- P1: AT=0, ST=0, FT=8
  WT = ST−AT = 0
  TAT = FT−AT = 8
  RT = ST−AT = 0

- P2: AT=1, ST=8, FT=12
  WT = 8−1 = 7
  TAT = 12−1 = 11
  RT = 8−1 = 7

- P4: AT=3, ST=12, FT=17
  WT = 12−3 = 9
  TAT = 17−3 = 14
  RT = 12−3 = 9

- P3: AT=2, ST=17, FT=26
  WT = 17−2 = 15
  TAT = 26−2 = 24
  RT = 17−2 = 15

Averages (SJF)

- Avg WT = (0 + 7 + 9 + 15) / 4 = **7.75**
- Avg TAT = (8 + 11 + 14 + 24) / 4 = **14.25**
- Avg RT = (0 + 7 + 9 + 15) / 4 = **7.75**

Compare with FCFS (from warm-up)

- FCFS averages: Avg WT = **8.75**, Avg TAT = **15.25**, Avg RT = **8.75**
- SJF improved each metric by **1.00** unit here:
  - WT: 8.75 → **7.75**

- TAT: 15.25 → **14.25**
- RT: 8.75 → **7.75**

Warm-Up C — Quantum effect (Round Robin)

For P1(AT=0,BT=10), P2(0,5), P3(0,2), q=2, no context cost.

1. **Simulate RR order for first 10 units.**
2. **What happens if q increases to 10?**

Solution:

Processes (all arrive at 0):
P1 (BT = 10), P2 (BT = 5), P3 (BT = 2)
Quantum q = 2, CPU has no context-switch overhead.

Step-by-step timeline (first 10 time units)

We give each ready process up to 2 units in turn.

Note: Remaining Time (rem)

- **t = 0–2:** P1 runs (P1 uses 2 → rem 8)
- **t = 2–4:** P2 runs (P2 uses 2 → rem 3)
- **t = 4–6:** P3 runs (P3 uses 2 → rem 0) → **P3 completes at t = 6**
- **t = 6–8:** P1 runs (P1 uses 2 → rem 6)
- **t = 8–10:** P2 runs (P2 uses 2 → rem 1)

So the **RR order for first 10 units** is:

```
Gantt (0→10):  | P1 (0-2) | P2 (2-4) | P3 (4-6) | P1 (6-8) | P2 (8-10) |
```

State at t = 10

Process	Original BT	Used so far	Remaining BT	Completed?	First start time
P1	10	4	6	No	0
P2	5	4	1	No	2
P3	2	2	0	**Yes** at 6	4

Response times (first start − arrival):

- P1: 0 − 0 = **0**
- P2: 2 − 0 = **2**
- P3: 4 − 0 = **4**
  Average RT (already known after first start): (0 + 2 + 4) / 3 = **2.0**

(Waiting time & turnaround for P1 and P2 cannot be finalized until they complete, but P3: FT=6, TAT=6, WT=TAT−BT=6−2=4.)

What happens if q increases to 10?

Set q = 10 (quantum large, >= largest burst).

- With q = 10, the first process in queue will get the CPU and run until it **finishes** (because its burst ≤ quantum).
- Since all arrive at t=0, ordering is by arrival (all same) → CPU runs **P1 (0–10)**, then **P2 (10–15)**, then **P3 (15–17)**.
- That behavior is effectively **FCFS** (non-preemptive) when q is large enough to cover whole bursts.

Consequences (small q vs large q):

- **Small q (e.g., 2):**
    - More frequent context switches (if cost exists).
    - Better **response time** for short jobs (they get CPU earlier).
    - More fairness and interactivity.
    - Higher overhead if context switching cost is nonzero.
- **Large q (e.g., 10):**
    - Fewer switches → lower overhead.
    - System behaves like **FCFS** → short jobs may wait longer behind long jobs.
    - Worse response time for jobs that are later in queue.

---

3) Graded Tasks — with Solutions

Task 1 — FCFS + Non-Preemptive Priority (tie-break safe)

Goal: Implement FCFS and **Priority (non-preemptive)** in one program:

- Handle **arrival times**; when CPU is idle, jump to next arrival.
- Priority: lower number = higher priority; tie-break by arrival, then id.
- Output: **Gantt timeline**, per-process **WT, TAT, RT**, and **averages**.

Task 2 — SJF (Non-preemptive) vs SRTF (Preemptive) + Response Time

Goal: One program that runs **both algorithms** and prints a side-by-side metrics comparison:

- Correct handling of **new arrivals while running**.
- **SRTF**: preempt on any arrival with smaller remaining time.
- Report **Avg WT/TAT/RT**, and **# of preemptions** for SRTF.

Reference Solution (C: sjf_srtf.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { char id[16]; int at, bt; } In;
typedef struct {
   char id[16]; int at, bt, rem;
   int start, first_start_seen, finish;
} P;

int all_done(P* a,int n){ for(int i=0;i<n;i++) if(a[i].rem>0) return 0; return 1; }
int pick_sjf(P* a,int n,int t){
   int idx=-1, best_bt=1e9, best_at=1e9;
   for(int i=0;i<n;i++) if(a[i].rem>0 && a[i].at<=t){
      if(a[i].bt<best_bt || (a[i].bt==best_bt && (a[i].at<best_at || (a[i].at==best_at &&
strcmp(a[i].id,a[idx].id)<0)))){
         best_bt=a[i].bt; best_at=a[i].at; idx=i;
      }
   }
   return idx;
}
int pick_srtf(P* a,int n,int t){
   int idx=-1, best_rem=1e9, best_at=1e9;
   for(int i=0;i<n;i++) if(a[i].rem>0 && a[i].at<=t){
      if(a[i].rem<best_rem || (a[i].rem==best_rem && (a[i].at<best_at || (a[i].at==best_at &&
strcmp(a[i].id,a[idx].id)<0)))){
         best_rem=a[i].rem; best_at=a[i].at; idx=i;
      }
   }
   return idx;
}
void copy_in(In*in, P*out,int n){ for(int i=0;i<n;i++){ strcpy(out[i].id,in[i].id); out[i].at=in[i].at;
out[i].bt=in[i].bt; out[i].rem=in[i].bt; out[i].start=-1; out[i].finish=-1; out[i].first_start_seen=0; } }

void run_sjf(In*in,int n){
   P a[n]; copy_in(in,a,n);
   int t=0; printf("SJF Gantt: ");
   while(!all_done(a,n)){
      int ready=0; for(int i=0;i<n;i++) if(a[i].rem>0 && a[i].at<=t){ ready=1; break; }
```

```
        if(!ready){ int next=1e9; for(int i=0;i<n;i++) if(a[i].rem>0 && a[i].at<next) next=a[i].at; t=next;
continue; }
        int idx=pick_sjf(a,n,t);
        if(!a[idx].first_start_seen){ a[idx].start=t; a[idx].first_start_seen=1; }
        printf("| %s(%d-%d) ", a[idx].id, t, t+a[idx].rem);
        t += a[idx].rem; a[idx].rem=0; a[idx].finish=t;
    }
    printf("|\n");
    double w=0,ta=0,r=0; printf("ID\tAT\tBT\tST\tFT\tWT\tTAT\tRT\n");
    for(int i=0;i<n;i++){
        int wt=(a[i].finish - a[i].at) - in[i].bt;
        int tat=a[i].finish - a[i].at;
        int rt=a[i].start - a[i].at;
        w+=wt; ta+=tat; r+=rt;
        printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", in[i].id, in[i].at, in[i].bt, a[i].start, a[i].finish, wt, tat,
rt);
    }
    printf("SJF Averages -> WT: %.2f  TAT: %.2f  RT: %.2f\n", w/n, ta/n, r/n);
}

void run_srtf(In*in,int n){
    P a[n]; copy_in(in,a,n);
    int t=0, cur=-1, preemptions=0; printf("SRTF Gantt: ");
    while(!all_done(a,n)){
        int ready=0; for(int i=0;i<n;i++) if(a[i].rem>0 && a[i].at<=t){ ready=1; break; }
        if(!ready){ int next=1e9; for(int i=0;i<n;i++) if(a[i].rem>0 && a[i].at<next) next=a[i].at; t=next;
cur=-1; continue; }
        int pick=pick_srtf(a,n,t);
        if(pick!=cur){ if(cur!=-1) printf("(%d) ", t); // cut segment
            cur=pick; printf("| %s(%d-", a[cur].id, t); preemptions++;
            if(!a[cur].first_start_seen){ a[cur].start=t; a[cur].first_start_seen=1; }
        }
        a[cur].rem--; t++;
        if(a[cur].rem==0){ printf("%d) ", t); a[cur].finish=t; cur=-1; }
    }
    printf("|\n");
    double w=0,ta=0,r=0; printf("ID\tAT\tBT\tST\tFT\tWT\tTAT\tRT\n");
    for(int i=0;i<n;i++){
        int wt=(a[i].finish - a[i].at) - in[i].bt;
        int tat=a[i].finish - a[i].at;
        int rt=a[i].start - a[i].at;
        w+=wt; ta+=tat; r+=rt;
        printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", in[i].id, in[i].at, in[i].bt, a[i].start, a[i].finish, wt, tat,
rt);
    }
    printf("SRTF Averages -> WT: %.2f  TAT: %.2f  RT: %.2f   Preemptions: %d\n", w/n, ta/n, r/n,
preemptions);
}

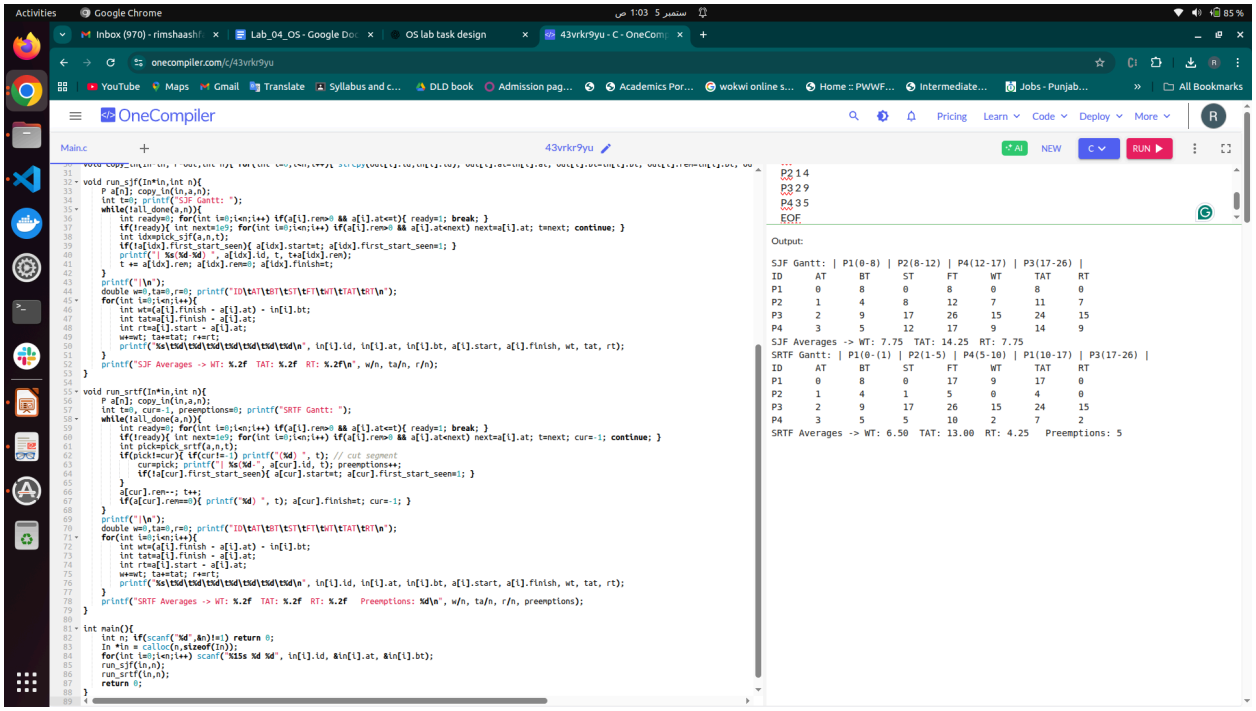
int main(){
    int n; if(scanf("%d",&n)!=1) return 0;
```

```
    In *in = calloc(n,sizeof(In));
    for(int i=0;i<n;i++) scanf("%15s %d %d", in[i].id, &in[i].at, &in[i].bt);
    run_sjf(in,n);
    run_srtf(in,n);
    return 0;
}
```

Run:

```
gcc sjf_srtf.c -o sch2 && ./sch2 <<EOF
4
P1 0 8
P2 1 4
P3 2 9
P4 3 5
EOF
```

What to observe: SRTF should yield lower **Avg WT/RT** than SJF, at cost of more preemptions.



Task 3 — Round Robin with Context-Switch Cost (+ fairness & throughput)

Goal: Implement **RR** with:

- **Time quantum q** (input).

- **Context switch cost cs**: upon **every switch** (including preemptions), add cs time units (CPU idle).
- Print **Gantt chart** showing switch gaps, **Avg WT/TAT/RT**, **#context switches**, **throughput**.

*(Optional extension for top students: add **aging** to a priority-ready queue, or simulate an **MLFQ**. If you want, I can add an MLFQ solution too.)*

Checkpoint Questions

1. Why can SRTF starve long jobs? How could you prevent it?
2. How does **context switch cost** change RR behavior as **q → 1** vs **q large**?
3. Which metric is most sensitive to **preemption** and why?
